

LAPORAN PRATIKUM STRUKTUR DATA

LAPORAN PRAKTIKUM PEKAN 5

Disusun Oleh:

Nama: Muhammad Aufa Rafiki

NIM: 2511531012

Dosen Pengampu: Dr. Wahyudi S.T.M.T

Asisten Praktikum: M. Galid



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, saya dapat menyelesaikan laporan praktikum ini dengan baik. Laporan ini disusun sebagai salah satu tugas dalam memahami konsep struktur data, khususnya Single Linked List (SLL) menggunakan bahasa pemrograman Java.

Dalam laporan ini, dibahas mengenai implementasi operasi dasar pada Single Linked List, seperti penambahan data, penghapusan data, serta pencarian data. Saya menyadari bahwa laporan ini masih memiliki kekurangan, oleh karena itu saya mengharapkan kritik dan saran yang membangun untuk perbaikan di masa yang akan datang.

Akhir kata, saya berharap laporan ini dapat memberikan manfaat bagi pembaca dan menambah wawasan dalam bidang struktur data.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Pratikum.....	1
1.3 Manfaat Pratikum	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori.....	2
2.2 NodeSLL_2511531012.....	2
2.3 TambahSLL_2511531012	3
2.4 HapusSLL_2511531012	5
2.5 PencarianSLL_2511531012	7
BAB III KESIMPULAN	8
DAFTAR PUSTAKA	9
Tugas	10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam ilmu komputer yang digunakan untuk mengorganisasi dan menyimpan data secara efisien. Salah satu struktur data yang sering digunakan adalah Linked List, khususnya Single Linked List (SLL).

Single Linked List adalah struktur data linear yang terdiri dari kumpulan node, di mana setiap node memiliki data dan pointer yang menunjuk ke node berikutnya. Berbeda dengan array, Linked List memiliki kelebihan dalam hal fleksibilitas penambahan dan penghapusan data.

Dalam praktikum ini, dilakukan implementasi berbagai operasi pada Single Linked List seperti menambah node, menghapus node, serta mencari data dalam list menggunakan bahasa pemrograman Java.

1.2 Tujuan Pratikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut:

1. Memahami konsep dasar Single Linked List
2. Mampu mengimplementasikan struktur Single Linked List dalam Java
3. Mengetahui cara menambahkan node (di depan, belakang, dan posisi tertentu)
4. Mengetahui cara menghapus node (head, akhir, dan posisi tertentu)
5. Mampu melakukan pencarian data dalam Linked List

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini antara lain:

1. Meningkatkan pemahaman tentang struktur data
2. Melatih kemampuan pemrograman Java
3. Memahami cara kerja pointer/referensi dalam Linked List
4. Dapat menerapkan konsep Linked List dalam pemecahan masalah

BAB II

PEMBAHASAN

2.1 Dasar teori

Struktur data merupakan cara untuk menyimpan dan mengelola data agar dapat diakses dan diproses secara efisien. Salah satu struktur data dinamis adalah Linked List, yaitu kumpulan elemen yang disebut node yang saling terhubung melalui pointer atau referensi. Setiap node pada Single Linked List (SLL) terdiri dari dua bagian, yaitu data dan pointer yang menunjuk ke node berikutnya, di mana node terakhir menunjuk ke null sebagai penanda akhir. Berbeda dengan array, Linked List tidak memerlukan alokasi memori yang berurutan sehingga lebih fleksibel dalam penambahan dan penghapusan data. Operasi dasar dalam Single Linked List meliputi traversal (penelusuran), insertion (penyisipan), deletion (penghapusan), dan searching (pencarian), yang semuanya dilakukan dengan cara menelusuri node satu per satu melalui pointer yang saling terhubung.

2.2 NodeSLL_2511531012

```
1 package pekan5_2511531012;
2
3 public class NodeSLL_2511531012 {
4     // node bagian data
5     int data_1012;
6     //pointer ke node berikutnya
7     NodeSLL_2511531012 next_1012;
8     //Konstruktor menginisialisasi node dengan data
9     public NodeSLL_2511531012(int data_1012) {
10         this.data_1012 = data_1012;
11         this.next_1012 = null;
12     }
13 }
14
```

Penjelasan Program:

1. Class ini adalah dasar dari Single Linked List karena merepresentasikan node (simpul).
2. `int data_1012;` → node bagian data, tempat menyimpan nilai/data
3. `NodeSLL_2511531012 next_1012;` → menyimpan referensi/pointer ke node berikutnya
4. Konstruktor menginisialisasi node dengan data
 - Saat objek dibuat, nilai langsung disimpan ke `data_1012`
 - `next_1012` otomatis null (belum terhubung)

2.3 TambahSLL_2511531012

```
1 package pekan5_2511531012;
2
3 public class TambahSLL_2511531012 {
4     public static NodeSLL_2511531012 insertAtFront_1012(NodeSLL_2511531012 head_1012, int value_1012) {
5         NodeSLL_2511531012 new_node_1012 = new NodeSLL_2511531012(value_1012);
6         new_node_1012.next_1012 = head_1012;
7         return new_node_1012;
8     }
9     //fungsi menambahkan node di akhir SLL
10    public static NodeSLL_2511531012 insertAtEnd_1012(NodeSLL_2511531012 head_1012, int value_1012) {
11        // buat sebuah node dengan nilai
12        NodeSLL_2511531012 newNode_1012 = new NodeSLL_2511531012(value_1012);
13        // jika list kosong maka node jadi head
14        if (head_1012 == null) {
15            return newNode_1012;
16        }
17        //simpan head ke variabel sementara
18        NodeSLL_2511531012 last_1012 = head_1012;
19        //telusuri ke node akhir
20        while (last_1012.next_1012 != null) {
21            last_1012 = last_1012.next_1012;
22        }
23        //ubah pointer
24        last_1012.next_1012 = newNode_1012;
25        return head_1012;
26    }
27    static NodeSLL_2511531012 GetNode_1012(int data_1012) {
28        return new NodeSLL_2511531012(data_1012);
29    }
30    static NodeSLL_2511531012 insertPos_1012(NodeSLL_2511531012 headNode_1012, int position_1012, int value_1012) {
31        NodeSLL_2511531012 head_1012 = headNode_1012;
32        if (position_1012 < 1)
33            System.out.print("Invalid position");
34        if (position_1012 == 1) {
35            NodeSLL_2511531012 new_node_1012 = new NodeSLL_2511531012(value_1012);
36            new_node_1012.next_1012 = head_1012;
37            return new_node_1012;
38        } else {
39            while (position_1012-- != 0) {
40                if (position_1012 == 1) {
41                    NodeSLL_2511531012 newNode_1012 = GetNode_1012(value_1012);
42                    newNode_1012.next_1012 = headNode_1012.next_1012;
43                    headNode_1012.next_1012 = newNode_1012;
44                    break;
45                }
46                headNode_1012 = headNode_1012.next_1012;
47            }
48        }
49        if (position_1012 != 1)
50            System.out.print("Posisi di luar jangkauan");
51        return head_1012;
52    }
53
54    public static void printList_1012(NodeSLL_2511531012 head_1012) {
55        NodeSLL_2511531012 curr_1012 = head_1012;
56        while (curr_1012.next_1012 != null) {
57            System.out.print(curr_1012.data_1012+"-->");
58            curr_1012 = curr_1012.next_1012;
59        }
60        if (curr_1012.next_1012 == null) {
61            System.out.print(curr_1012.data_1012);
62            System.out.println();
63        }
64    }
65    public static void main(String[] args) {
66        //buat linked list 2->3->5->6
67        NodeSLL_2511531012 head_1012 = new NodeSLL_2511531012(2);
68        head_1012.next_1012 = new NodeSLL_2511531012(3);
69        head_1012.next_1012.next_1012 = new NodeSLL_2511531012(5);
70        head_1012.next_1012.next_1012.next_1012 = new NodeSLL_2511531012(6);
71        //cetak list asli
72        System.out.print("Senarai berantai awal: ");
73        printList_1012(head_1012);
74        //tambahkan node baru di depan
75        System.out.print("Tambah 1 simpul di depan: ");
76        int data_1012 = 1;
77        head_1012 = insertAtFront_1012(head_1012, data_1012);
78
79        printList_1012(head_1012);
80
81        System.out.print("tambah 1 simpul di belakang: ");
82        int data2_1012 = 7;
83        head_1012 = insertAtEnd_1012(head_1012, data2_1012);
84
85        printList_1012(head_1012);
86
87        System.out.print("tambah 1 simpul ke data ke 4: ");
88        int data3_1012 = 4;
89        int pos_1012 = 4;
90        head_1012 = insertPos_1012(head_1012, pos_1012, data3_1012);
91
92        printList_1012(head_1012);
93    }
94 }
95
```

Penjelasan Program:

1. Class ini berfungsi untuk menambahkan node ke dalam Linked List.
2. Method insertAtFront_1012 → menambahkan node di awal(head), dengan cara: Membuat node baru, node baru menunjuk ke head lama, head dipindahkan ke node baru
3. Method insertAtEnd_1012 → Menambahkan node di akhir list, dengan cara:
 - Buat node baru
 - jika list kosong maka node jadi head
 - Jika tidak:
 - Telusuri sampai node terakhir
 - Hubungkan node terakhir ke node baru
4. Method GetNode_1012 → Membuat node baru (fungsi pembantu), dengan cara: langsung mengembalikan objek node baru
5. Method insertPos_1012 → Menambahkan node di posisi tertentu, dengan cara:
 - Jika posisi < 1 → tidak valid
 - Jika posisi = 1 → sama seperti insert depan
 - Jika posisi lain:
 - Traversing hingga posisi sebelum target
 - Buat node baru
 - Atur pointer:
node baru → next node lama
node sebelumnya → node baru
 - Jika posisi melebihi panjang list → tampil pesan error
6. Method printList_1012 → Menampilkan isi Linked List, dengan cara: mulai dari head, cetak data selama belum null, bergerak ke node berikutnya
7. Method main() → Sebagai program utama untuk pengujian.
 - buat linked list 2→3→5→6
 - cetak list asli dengan → printList_1012(head_1012);
 - tambahkan node baru di depan → head_1012=insertAtFront_1012(head_1012, data_1012); lalu cetak list → printList_1012(head_1012);
 - tambahkan node baru di dibelakang → head_1012= insertAtEnd_1012 (head_1012, data_1012); lalu cetak list → printList_1012(head_1012);
 - tambahkan node baru di data/list ke → head_1012= insertPos_1012(head_1012, data_1012); lalu cetak list → printList_1012(head_1012);

Output:

```
Senarai berantai awal: 2-->3-->5-->6
Tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data ke 4: 1-->2-->3-->4-->5-->6-->7
```

2.4 HapusSLL_2511531012

```
1 package pekan5_2511531012;
2
3 public class HapusSLL_2511531012 {
4
5     public static NodeSLL_2511531012 deleteHead_1012(NodeSLL_2511531012 head_1012) {
6
7         if (head_1012 == null)
8             return null;
9         head_1012 = head_1012.next_1012;
10
11         return head_1012;
12     }
13     public static NodeSLL_2511531012 removeLastNode_1012(NodeSLL_2511531012 head_1012) {
14
15         if (head_1012 == null) {
16             return null;
17         }
18         if (head_1012.next_1012 == null) {
19             return null;
20         }
21
22         NodeSLL_2511531012 secondLast_1012 = head_1012;
23         while (secondLast_1012.next_1012.next_1012 != null) {
24             secondLast_1012 = secondLast_1012.next_1012;
25         }
26
27         secondLast_1012.next_1012 = null;
28         return head_1012;
29     }
30
31     public static NodeSLL_2511531012 deleteNode_1012(NodeSLL_2511531012 head_1012, int position_1012) {
32         NodeSLL_2511531012 temp_1012 = head_1012;
33         NodeSLL_2511531012 prev_1012 = null;
34
35         if (temp_1012 == null)
36             return head_1012;
37
38         if (position_1012 == 1) {
39             head_1012 = temp_1012.next_1012;
40             return head_1012;
41         }
42
43         for (int i = 1; temp_1012 != null && i < position_1012; i++) {
44             prev_1012 = temp_1012;
45             temp_1012 = temp_1012.next_1012;
46         }
47
48         if (temp_1012 != null) {
49             prev_1012.next_1012 = temp_1012.next_1012;
50         } else {
51             System.out.println("Data tidak ada");
52             return head_1012;
53         }
54
55     public static void printList_1012(NodeSLL_2511531012 head_1012) {
56         NodeSLL_2511531012 curr_1012 = head_1012;
57         while (curr_1012.next_1012 != null) {
58             System.out.print(curr_1012.data_1012+ "-->");
59             curr_1012 = curr_1012.next_1012;
60         }
61         if (curr_1012.next_1012 == null) {
62             System.out.print(curr_1012.data_1012);
63         }
64         System.out.println();
65     }
66
67     public static void main (String[] args) {
68
69         NodeSLL_2511531012 head_1012 = new NodeSLL_2511531012(1);
70         head_1012.next_1012 = new NodeSLL_2511531012(2);
71         head_1012.next_1012.next_1012 = new NodeSLL_2511531012(3);
72         head_1012.next_1012.next_1012.next_1012 = new NodeSLL_2511531012(4);
73         head_1012.next_1012.next_1012.next_1012.next_1012 = new NodeSLL_2511531012(5);
74         head_1012.next_1012.next_1012.next_1012.next_1012.next_1012 = new NodeSLL_2511531012(6);
75
76         System.out.println("List awal: ");
77         printList_1012(head_1012);
78
79         head_1012 = deleteHead_1012(head_1012);
80         System.out.println("List setelah head dihapus: ");
81         printList_1012(head_1012);
82
83         head_1012 = removeLastNode_1012(head_1012);
84         System.out.println("List setelah simpul terakhir di hapus: ");
85         printList_1012(head_1012);
86
87         int position_1012 = 2;
88         head_1012 = deleteNode_1012(head_1012, position_1012);
89         System.out.println("List setelah posisi 2 dihapus: ");
90         printList_1012(head_1012);
91     }
92 }
93
94 }
```


Penjelasan Program:

1. Class ini digunakan untuk menghapus node.
2. Method deleteHead_1012 → Menghapus node pertama, dengan cara: jika SLL kosong, head digeser ke node berikutnya, return head baru.
3. Method removeLastNode_1012 → Menghapus node terakhir, dengan cara:
 - Jika list kosong → return null
 - Jika list hanya 1 node → hapus node dan return null, jadi kosong
 - Jika lebih:
 - Cari node sebelum terakhir
 - Set pointer ke null
4. Method deleteNode_1012 → Menghapus node pada posisi tertentu. Dengan cara:
 - Jika linked list null → return head
 - Jika posisi = 1 → hapus head
 - Lakukan perulangan, jika posisi tidak sama dengan null, bernilai benar maka hapus node. Jika tidak cetak data tidak ditemukan.
5. Method printList_1012 → Menampilkan isi list dari awal sampai akhir.
6. Method main()
 - Buat SLL 1→2→3→4→5→6
 - Cetak list → printList_1012(head_1012);
 - Hapus head → head_1012 = deleteHead_1012(head_1012); lalu cetak list
 - Hapus node terakhir → head_1012 = removeLastNode_1012 (head_1012); lalu cetak list
 - Hapus node pada posisi 2 → head_1012 = deleteNode_1012 (head_1012); lalu cetak list

Output:

```
List awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

2.5 PencarianSLL_2511531012

```
1 package pekan5_2511531012;
2
3 public class PencarianSLL_2511531012 {
4     static boolean searchKey_1012(NodeSLL_2511531012 head_1012, int key_1012) {
5         NodeSLL_2511531012 curr_1012 = head_1012;
6         while (curr_1012 != null) {
7             if(curr_1012.data_1012 == key_1012)
8                 return true;
9             curr_1012 = curr_1012.next_1012;
10        }
11        return false;
12    }
13    public static void traversal_1012(NodeSLL_2511531012 head_1012) {
14
15        NodeSLL_2511531012 curr_1012 = head_1012;
16
17        while (curr_1012 != null) {
18            System.out.print(" " + curr_1012.data_1012);
19            curr_1012 = curr_1012.next_1012;
20        }
21        System.out.println();
22    }
23
24    public static void main(String[] args) {
25        NodeSLL_2511531012 head_1012 = new NodeSLL_2511531012(14);
26        head_1012.next_1012 = new NodeSLL_2511531012(21);
27        head_1012.next_1012.next_1012 = new NodeSLL_2511531012(13);
28        head_1012.next_1012.next_1012.next_1012 = new NodeSLL_2511531012(30);
29        head_1012.next_1012.next_1012.next_1012.next_1012 = new NodeSLL_2511531012(10);
30        System.out.print("Penelusuran SLL");
31        traversal_1012(head_1012);
32
33        int key_1012=30;
34        System.out.print("cari data "+key_1012+" = ");
35        if (searchKey_1012(head_1012, key_1012))
36            System.out.println("ketemu");
37        else System.out.println("tidak ada");
38    }
39 }
40
41 }
42 }
```

Penjelasan program:

1. Class ini digunakan untuk pencarian dan traversal data.
2. Method searchKey_1012 → Mencari apakah suatu nilai ada dalam list. Dengan cara:
 - Mulai dari head
 - Bandingkan setiap node:
 - Jika sama → return true
 - Jika sampai akhir tidak ditemukan → return false
3. Method traversal_1012 → Menampilkan seluruh isi Linked List, dengan cara: Menelusuri dari head sampai null sambil mencetak data
4. Method main()
 - Membuat list: 14→21→13→30→10
 - Menampilkan list dengan : traversal_1012(head_1012);
 - Mencari data (misalnya 30) dengan: searchKey_1012(head_1012, key_1012)
 - Menampilkan hasil:
 - “ketemu” jika ada
 - “tidak ada” jika tidak ditemukan

Output:

```
Penelusuran SLL 14 21 13 30 10
cari data 30 = ketemu
```

BAB III

KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa Single Linked List merupakan struktur data dinamis yang terdiri dari kumpulan node yang saling terhubung melalui pointer atau referensi. Setiap node memiliki dua komponen utama, yaitu data dan pointer yang menunjuk ke node berikutnya, sehingga memungkinkan data disusun secara fleksibel tanpa harus berada pada lokasi memori yang berurutan. Melalui implementasi program menggunakan bahasa Java, operasi dasar seperti penambahan node di awal, di akhir, maupun pada posisi tertentu dapat dilakukan dengan baik sesuai dengan konsep yang telah dipelajari.

Selain itu, operasi penghapusan node, pencarian data, serta penelusuran (traversal) juga dapat dijalankan dengan cara mengatur hubungan antar node melalui pointer. Hal ini menunjukkan bahwa Linked List memiliki keunggulan dalam hal fleksibilitas dan efisiensi dalam manipulasi data dibandingkan struktur data statis. Dengan demikian, pemahaman mengenai Single Linked List sangat penting sebagai dasar dalam mempelajari struktur data yang lebih kompleks serta dalam pengembangan program yang membutuhkan pengelolaan data secara dinamis dan terstruktur.

DAFTAR PUSTAKA

- [1] I. Liem, *Draft Diktat Struktur Data*, Edisi 2008. Bandung, Indonesia: Program Studi Teknik Informatika, Institut Teknologi Bandung, 2008.

Tugas

Pasien_2511531012

```
1 package pekan5_2511531012;
2
3 public class Pasien_2511531012 {
4     private String namaPasien_1012;
5     private String penyakit_1012;
6     private int nomorAntrian_1012;
7     private Pasien_2511531012 next_1012;
8
9     public Pasien_2511531012(String namaPasien_1012, String penyakit_1012, int nomorAntrian_1012) {
10         this.namaPasien_1012 = namaPasien_1012;
11         this.penakit_1012 = penyakit_1012;
12         this.nomorAntrian_1012 = nomorAntrian_1012;
13         this.next_1012 = null;
14     }
15
16     // Getter
17     public String getNamaPasien_1012() { return namaPasien_1012; }
18     public String getPenyakit_1012() { return penyakit_1012; }
19     public int getNomorAntrian_1012() { return nomorAntrian_1012; }
20     public Pasien_2511531012 getNext_1012() { return next_1012; }
21
22     // Setter
23     public void setNamaPasien_1012(String namaPasien_1012) { this.namaPasien_1012 = namaPasien_1012; }
24     public void setPenyakit_1012(String penyakit_1012) { this.penakit_1012 = penyakit_1012; }
25     public void setNomorAntrian_1012(int nomorAntrian_1012) { this.nomorAntrian_1012 = nomorAntrian_1012; }
26     public void setNext_1012(Pasien_2511531012 next_1012) { this.next_1012 = next_1012; }
27 }
28 }
```

Penjelasan program:

1. Class ini berfungsi sebagai node dalam struktur Single Linked List yang menyimpan data pasien.
2. Atribut:
 - namaPasien_1012 → menyimpan nama pasien
 - penyakit_1012 → menyimpan keluhan/penyakit pasien
 - nomorAntrian_1012 → menyimpan nomor antrian pasien
 - next_1012 → pointer yang menunjuk ke node berikutnya
3. Constructor:
 - Digunakan untuk menginisialisasi data pasien saat node dibuat
 - Pointer next_1012 di-set null karena belum terhubung ke node lain
4. Getter: → Digunakan untuk mengambil nilai atribut (nama, penyakit, nomor, dan next)
5. Setter:
 - Digunakan untuk mengubah nilai atribut
 - setNext_1012() sangat penting karena digunakan untuk menghubungkan node satu dengan yang lain dalam linked list

RumahSakit_2511531012

```
1 package pekan5_2511531012;
2
3 import java.util.Scanner;
4
5 public class RumahSakit_2511531012 {
6     private static Pasien_2511531012 head_1012 = null; //node pertama (pasien terdepan)
7     private static Pasien_2511531012 tail_1012 = null; //node terakhir (pasien paling belakang)
8     private static int counter_1012 = 0; //menyimpan nomor antrian (auto increment)
9
10    // Method untuk menambah pasien (Insert at Tail / FIFO)
11    public static void daftarPasien_1012(String namaPasien_1012, String penyakit_1012) {
12        // Menambah nomor antrian secara otomatis
13        counter_1012++;
14
15        // Membuat node baru
16        Pasien_2511531012 pasienBaru_1012 = new Pasien_2511531012(namaPasien_1012, penyakit_1012, counter_1012);
17        // Jika list kosong, node baru jadi head dan tail
18        if (head_1012 == null) {
19            head_1012 = tail_1012 = pasienBaru_1012;
20        } else {
21            // Menghubungkan node terakhir ke node baru
22            tail_1012.setNext_1012(pasienBaru_1012);
23            // Memindahkan tail ke node baru
24            tail_1012 = pasienBaru_1012;
25        }
26        System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: "
27            + pasienBaru_1012.getNomorAntrian_1012());
28    }
29
30    // Method untuk memanggil pasien (Delete Head)
31    public static void panggilPasien_1012() {
32        // Jika list kosong, tidak ada pasien
33        if (head_1012 == null) {
34            System.out.println("Antrian kosong!");
35            return;
36        }
37    }
38
39    package pekan5_2511531012;
40
41    import java.util.Scanner;
42
43    public class RumahSakit_2511531012 {
44        private static Pasien_2511531012 head_1012 = null;
45        private static Pasien_2511531012 tail_1012 = null;
46        private static int counter_1012 = 0;
47
48        // Method untuk menambah pasien (Insert at Tail / FIFO)
49        public static void daftarPasien_1012(String namaPasien_1012, String penyakit_1012) {
50            // Menambah nomor antrian secara otomatis
51            counter_1012++;
52
53            // Membuat node baru
54            Pasien_2511531012 pasienBaru_1012 = new Pasien_2511531012(namaPasien_1012, penyakit_1012, counter_1012);
55            // Jika list kosong, node baru jadi head dan tail
56            if (head_1012 == null) {
57                head_1012 = tail_1012 = pasienBaru_1012;
58            } else {
59                // Menghubungkan node terakhir ke node baru
60                tail_1012.setNext_1012(pasienBaru_1012);
61                // Memindahkan tail ke node baru
62                tail_1012 = pasienBaru_1012;
63            }
64            System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: "
65                + pasienBaru_1012.getNomorAntrian_1012());
66        }
67
68        // Method untuk memanggil pasien (Delete Head)
69        public static void panggilPasien_1012() {
70            // Jika list kosong, tidak ada pasien
71            if (head_1012 == null) {
72                System.out.println("Antrian kosong!");
73                return;
74            }
75        }
76    }
```

```

36     }
37     // Menampilkan data pasien terdepan
38     System.out.println("Memanggil pasien:");
39     System.out.println("No: " + head_1012.getNomorAntrian_1012());
40     System.out.println("Nama: " + head_1012.getNamaPasien_1012());
41     System.out.println("Keluhan: " + head_1012.getPenyakit_1012());
42
43     // Menggeser head ke node berikutnya (hapus node depan)
44     head_1012 = head_1012.getNext_1012();
45
46     // Jika setelah dihapus list kosong, tail juga harus null
47     if (head_1012 == null) {
48         tail_1012 = null;
49     }
50 }
51
52 // Method untuk menampilkan semua antrian (Traversal)
53 public static void tampilkanAntrian_1012() {
54
55     // Jika list kosong
56     if (head_1012 == null) {
57         System.out.println("Antrian kosong!");
58         return;
59     }
60
61     Pasien_2511531012 temp_1012 = head_1012;
62     int posisi_1012 = 1;
63
64     // Menelusuri linked list dari head sampai null
65     while (temp_1012 != null) {
66         System.out.println("Posisi: " + posisi_1012 +
67             " | No: " + temp_1012.getNomorAntrian_1012() +
68             " | Nama: " + temp_1012.getNamaPasien_1012() +
69             " | Keluhan: " + temp_1012.getPenyakit_1012());
70
71         // Pindah ke node berikutnya
72         temp_1012 = temp_1012.getNext_1012();
73         posisi_1012++;
74     }
75 }
76
77 // Method untuk mencari pasien (Search)
78 public static void cariPasien_1012(String namaPasien_1012) {
79     Pasien_2511531012 temp_1012 = head_1012;
80     boolean ditemukan_1012 = false;
81     // Traversal untuk mencari data
82     while (temp_1012 != null) {
83         // Perbandingan tanpa membedakan huruf besar/kecil
84         if (temp_1012.getNamaPasien_1012().equalsIgnoreCase(namaPasien_1012)) {
85
86             System.out.println("Pasien ditemukan!");
87             System.out.println("No: " + temp_1012.getNomorAntrian_1012() +
88                 " | Nama: " + temp_1012.getNamaPasien_1012() +
89                 " | Keluhan: " + temp_1012.getPenyakit_1012());
90             ditemukan_1012 = true;
91             break;
92         }
93         // Pindah ke node berikutnya
94         temp_1012 = temp_1012.getNext_1012();
95     }
96     // Jika tidak ditemukan
97     if (!ditemukan_1012) {
98         System.out.println("Pasien tidak ditemukan!");
99     }
100 }
101
102 // Method untuk cek status antrian
103 public static void cekStatusAntrian_1012() {
104     // Jika list kosong
105     if (head_1012 == null) {

```

```

106         System.out.println("Antrian kosong!");
107         return;
108     }
109     int jumlah_1012 = 0;
110     Pasien_2511531012 temp_1012 = head_1012;
111     // Menghitung jumlah node dalam linked list
112     while (temp_1012 != null) {
113         jumlah_1012++;
114         temp_1012 = temp_1012.getNext_1012();
115     }
116     // Menampilkan hasil
117     System.out.println("Jumlah pasien: " + jumlah_1012);
118     System.out.println("Pasien terdepan:");
119     System.out.println("No: " + head_1012.getNomorAntrian_1012() +
120         " | Nama: " + head_1012.getNamaPasien_1012() +
121         " | Keluhan: " + head_1012.getPenyakit_1012());
122 }
123
124 public static void main(String[] args) {
125     Scanner input_1012 = new Scanner(System.in);
126     int pilihan_1012;
127
128     do {
129         System.out.println("\n=== Antrian Rumah Sakit NIM: 2511531012 ===");
130         System.out.println("1. Daftarkan Pasien");
131         System.out.println("2. Panggil Pasien");
132         System.out.println("3. Tampilkan Antrian");
133         System.out.println("4. Cari Pasien");
134         System.out.println("5. Cek Status Antrian");
135         System.out.println("6. Keluar");
136         System.out.print("Pilihan: ");
137         pilihan_1012 = input_1012.nextInt();
138         input_1012.nextLine();
139
140         switch (pilihan_1012) {
141             case 1:
142                 System.out.print("Masukkan Nama Pasien : ");
143                 String nama_1012 = input_1012.nextLine();
144                 System.out.print("Masukkan Keluhan : ");
145                 String keluhan_1012 = input_1012.nextLine();
146                 daftarPasien_1012(nama_1012, keluhan_1012);
147                 break;
148             case 2:
149                 panggilPasien_1012();
150                 break;
151             case 3:
152                 tampilkanAntrian_1012();
153                 break;
154             case 4:
155                 System.out.print("Masukkan nama pasien: ");
156                 String cari_1012 = input_1012.nextLine();
157                 cariPasien_1012(cari_1012);
158                 break;
159             case 5:
160                 cekStatusAntrian_1012();
161                 break;
162             case 6:
163                 System.out.println("Terima kasih!");
164                 break;
165             default:
166                 System.out.println("Pilihan tidak valid!");
167         }
168     } while (pilihan_1012 != 6);
169     input_1012.close();
170 }
171 }
172 }

```

Penjelasan Program:

1. Class ini berfungsi untuk mengelola antrian pasien menggunakan Single Linked List.
2. Atribut
 - head_1012 → menunjuk node pertama (pasien terdepan)
 - tail_1012 → menunjuk node terakhir (pasien paling belakang)
 - counter_1012 → menyimpan nomor antrian (auto increment)
3. Method daftarPasien_1012() → Method ini digunakan untuk menambahkan pasien ke antrian.

- counter_1012 bertambah → nomor antrian otomatis
 - Membuat node baru (pasienBaru_1012)
 - Jika antrian kosong:
 - head dan tail menunjuk ke node baru
 - Jika tidak kosong:
 - Node baru ditambahkan di belakang (tail.next)
 - tail dipindahkan ke node baru
4. Method panggilPasien_1012() → Method ini digunakan untuk melayani pasien pertama dalam antrian.
- Jika head == null → antrian kosong
 - Menampilkan data pasien terdepan
 - Menghapus node depan dengan cara: head = head.next
 - Jika setelah dihapus kosong → tail = null
5. Method tampilkanAntrian_1012() → Method ini digunakan untuk menampilkan semua pasien.
- Jika antrian kosong → tampilkan pesan
 - Gunakan variabel temp_1012 untuk traversal
 - Loop dari head sampai null
 - Menampilkan:
 - posisi
 - nomor antrian
 - nama
 - keluhan
6. Method cariPasien_1012() → Method ini digunakan untuk mencari pasien berdasarkan nama.
- Traversal dari head
 - Bandingkan nama menggunakan: equalsIgnoreCase()
 - Jika ditemukan → tampilkan data
 - Jika tidak → tampilkan pesan tidak ditemukan
7. Method cekStatusAntrian_1012() → Method ini digunakan untuk melihat kondisi antrian.
- Jika kosong → tampilkan pesan
 - Hitung jumlah node dengan traversal
 - Tampilkan:
 - jumlah pasien
 - data pasien terdepan (head)

8. Method main()

- Menampilkan menu pilihan
- Input pilihan user
- Menggunakan switch-case untuk menjalankan:
 - tambah pasien
 - panggil pasien
 - tampilkan antrian
 - cari pasien
 - cek status
- Program berjalan berulang (do-while) sampai user memilih keluar

Output:

```
=== Antrian Rumah Sakit NIM: 2511531012 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : budi
Masukkan Keluhan      : demam
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511531012 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : andi
Masukkan Keluhan      : batuk
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511531012 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien : santi
Masukkan Keluhan      : flu
Pasien berhasil didaftarkan! Nomor Antrian: 3
```

```

1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 3
Posisi: 1 | No: 1 | Nama: budi | Keluhan: demam
Posisi: 2 | No: 2 | Nama: andi | Keluhan: batuk
Posisi: 3 | No: 3 | Nama: santi | Keluhan: flu

=== Antrian Rumah Sakit NIM: 2511531012 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah pasien: 3
Pasien terdepan:
No: 1 | Nama: budi | Keluhan: demam

=== Antrian Rumah Sakit NIM: 2511531012 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Memanggil pasien:
No: 1
Nama: budi
Keluhan: demam

=== Antrian Rumah Sakit NIM: 2511531012 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 3
Posisi: 1 | No: 2 | Nama: andi | Keluhan: batuk
Posisi: 2 | No: 3 | Nama: santi | Keluhan: flu

=== Antrian Rumah Sakit NIM: 2511531012 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan nama pasien: aufa
Pasien tidak ditemukan!

=== Antrian Rumah Sakit NIM: 2511531012 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Terima kasih!

```